四、IO 及进程线程部分（余宜宸老师）

4.1 对于文件 IO 你是怎么理解的

文件 IO 是系统提供的操作文件的函数接口，可以从封装形式、实现原理、优缺点 3 个方面来理解：

1）封装形式：文件 IO 封装了操作文件必要的函数接口，打开 open()，关闭 close()，读写 read()，定位 lseek()，足以实现基本的文件操作；对于复杂操作，需要用户自己写函数逻辑来实现；

2）实现原理：文件 IO 是 Linux 系统提供的函数接口，这些函数本身会依赖虚拟文件系统来访问磁盘数据，因为有虚拟文件系统和底下的驱动程序存在，文件 IO 就可以间接操作到各种磁盘下的文件数据，这种操作往往是没有缓冲的，即调用即操作；

3）优缺点：文件 IO 的优点是操作直接，屏蔽了磁盘格式差异；函数接口少，接口封装简洁易于使用；缺点是使用文件 IO 的程序依赖操作系统；并且不适用于循环反复读写数据的情况；复杂的读写方式需要用户自己设计，比如向文件指定位置插入一段数据就没有提供相关函数。

4.2 进程开销过大，有什么方法可以降低开销

首先进程的开销有 2 个方面，一个是对内存的开销（程序运行占用的内存空间），一个是对 CPU 的开销（进程过多的占用了 CPU 时间）。所以降低开销可以从 2 个方面入手，时间和空间；

降低进程占用的内存空间，可以考虑以下细节：

1）对于多进程而言，使用函数 vfork()创建进程可以让子进程先执行并共享父进程的数据，这可以明显降低单位时间段内因父子进程拷贝带来的开销；后续 fork 采用了的写时拷贝（Copy-On-Write，COW）技术，该技术允许在创建子进程时，父进程和子进程共享相同的物理内存空间，直到任一进程尝试修改其内存页时，才会为修改者创建该内存页的副本；

2）对于大量的配置型参数，使用宏定义或者配置文件的方式，避免用户交互式输入带来的时间和空间开销；

3）对于使用频率不是很高的外部资源（数据库、文件等），采用即使用即释放的策略避免数据长期存在于内存中；

4）优化代码结构以节省空间。比如：循环的方式比递归方式节省栈空间，变长的链表比固定的数组节约内存空间；

降低进程的 CPU 开销，可以考虑如下细节：

1）在面向对象化编程过程中采用重用对象策略：如果一个对象需要不断的构造析构，可以用对象池来管理。这种方式属于典型的用空间换时间；

华清远见武汉中心嵌入式学科面试宝典

2）对于需要复杂配置/计算后才能实现的功能，应该将配置/计算结果进行存储。比如：界面刷新时，重新绘制不如保留绘制对象的空间；频繁使用短连接不如维持长连接，这样可以避免重复的 3 次握手带来的消耗；

3）将一些图像处理任务交给外部 GPU（显卡）来计算，也能大幅度节省 CPU 的时间；

4）避免不必要的轮序/检测/扫描逻辑。比如：用 sleep(~0)替换 while(1)使得进程进入休眠状态而不是忙等状态；再比如使用信号通知进程事件而不是进程轮询检测时间也能节省 CPU 时间；

4.3 进程间的通信有哪些？各自的特点

进程间通信分为 3 种：数据交换、信号通知、互斥同步。

交换数据就要求 2 个进程访问同一个资源（内核空间或者公有文件），与之相关的通信方式有：管道、消息队列、共享内存；

信号通知就要求 2 个进程能互相通知对方以告知事件，使用函数 kill()及 signal()能够实现 2 个进程相互通知的效果，不过信号表示的含义有限，并且信号中不但能携带参数，导致其只能实现进程间基本事件的通知效果；

互斥同步强调的是进程间运行关系，可以使用信号量集合来实现，不过需要实事先约定信号量集合中信号量值的意义；

传统进程间通信方式包括：无名管道(pipe)、有名管道(fifo)、信号；System V 下进程间通信方式包括：消息队列、信号灯、共享内存；还有专门用于跨主机进程间通信的 BSD 套接字（socket）

无名管道	1）无名管道是半双工的； 2）只能用于亲缘进程之间交换数据； 3）空管道造成读阻塞，管道满导致写阻塞； 4）只能一对一通信；
有名管道	1）有名管道也是半双工的； 2）允许任意进程间进行通信； 3）只有在管道有读端时，往管道中写数据才有意义。否则，向管道写数据的进程会接收到内核发出来的 SIGPIPE 信号； 4）只能一对一通信。
信号	1）信号是 Unix 系统中使用的最古老的进程间通信的方法之一； 2）信号只能通知某一进程发生了某一种标准事件，不能携带数据信息；

华清远见武汉中心嵌入式学科面试宝典

	3）信号是一种异步通信方式，主要作用是通知目标进程发生的标准事件； 4）收到信号的进程有 3 中处理方式：默认处理、忽略信号、自定义信号处理函数； 5）信号源可以是：异常、其他进程、终端的中断(Ctrl-C,Ctrl+\等)、作业的控制(前台，后台进程的管理等)、分配额问题(cpu 超时或文件过大等)、内核通知(例如 I/O 就绪等)、报警(计时器)。
消息队列	1）消息队列就是消息的一个链表，它允许一个或者多个进程向它写消息； 2）消息队列本质是 Linux 维护的一个消息队列向量表； 3）消息队列克服了信号传递信息少，管道只能支持无格式字节流和缓冲区受限的缺点； 4）1 个消息队列就可以实现多对多通信
共享内存	1）共享内存是一种最为高效的进程间通信方式，不需要任何函数即可操作数据； 2）为了在多个进程间交换信息，内核专门留出了一块内存区，可以由需要访问的进程将其映射到自己的私有地址空间。
信号量集合	1）内核管理的一堆计数器，标识了共享资源的访问资格数； 2）由进程去操作指定集合中指定的信号量，只有成功占用了信号量才能访问共享资源；
套接字（socket）	1）主要用于不同主机间的进程通信； 2）一个套接口可以看做是进程间通信的端点(endpoint)，每个套接口的名字是唯一的； 3）套接字需要多种框架来维护，比如 IO 多路复用或者线程池； 4）套接字通信依赖完备的通信协议；

具体的通信方式对比如下表：

	本质	功能意义	通信方向	一对多	特点
无名管道	父子进程以文件 IO 的形式访问同一段内核空间	交换数据	创立后单向	1 对 1	简单，自带读写阻塞效果
有名管道	2 个进程读写文件系统中的同一个文件	交换数据	创立后单向	1 对 1	任意进程间，自带读写阻塞效果
消息队列	多个进程访问内核管理的同一个队列	交换数据	双向	多对多	一个队列实现多对多通信
共享内存	多个进程访问内核管理的同一片内存区域	交换数据	双向可设置	1 对多	多对多通信，效率最高，需要依赖同步

华清远见武汉中心嵌入式学科面试宝典

信号	一个进程通过系统通知另外一个进程标准事件	通知事件	双向	1 对 1 1 对多	异步，不定期发送信号都能得到处理
信号量集合	用内核管理的计数器标识共享资源的访问资格	控制进程访问资源的权利	双向	多对多	可以定制同步及互斥机制

4.4 什么是进程、线程、管程、程序、作业，以及什么实际场景使用他们

进程（Process）：是计算机中的程序关于某数据集合上的一次运行活动，是系统进行资源分配和调度的基本单位，是操作系统结构的基础。简言之运行的程序就是进程。

线程（Thread）：是操作系统能够进行运算调度的最小单位。它被包含在进程之中，是进程中的实际运作单位。

管程（Monitor）：是一种高级的同步机制，用于管理共享资源，管程定义了一个数据结构和能为并发进程所执行（在该数据结构上）的一组操作，这组操作能同步进程和改变管程中的数据。管程内的操作是互斥的，即一次只允许一个进程进入管程执行操作。

程序（Program）：一组有序指令的集合，静态概念，用于解决特定信息处理任务。

作业（Job）：用户提交给计算机系统的任务集合，包括程序、数据和作业说明书，主要在批处理系统中使用。

实际场景：首先明确多进程和多线程都能“同时实现多个任务”。假设要同时实现任务 A 和任务 B，在实现的过程，分如下两种情况：

如果任务 A 和任务 B 都需要自己拥有独立的资源才能往下执行那就必须是多进程。（任务 A 是盖大楼需要工人及建材，任务 B 是建桥梁也要工人及建材，那么这两个任务就需要两倍的资源才能同时进行）

如果任务 A 和任务 B 都公用一套资源就能往下执行那就首选多线程。（任务 A 是烧开水泡茶需要水壶及加热工具，任务 B 是烧开水洗碗也需要水壶及加热工具，那么两个任务共用一套资源就能完成，没必要使用多进程）

管程：它特别适用于生产者-消费者问题、读者-写者问题等经典并发场景，在这些场景中，管程能够确保多个进程或线程在访问共享资源时的正确性和效率。通过管程，程序员可以不必显式地编写复杂的同步代码，从而简化了并发程序的设计和实现

作业：批处理系统中作业的使用场景主要集中在批量数据处理任务，如科学计算、数据分析、报表生成等。这些任务通常不需要用户即时交互，而是按照预定的顺序和步骤执行。

华清远见武汉中心嵌入式学科面试宝典

4.5 多线程与多进程的区别

相同点：

1）线程和进程都是处理多任务的，使得一份代码能同时执行多个任务；

2）程序复杂性上升，任务之间如果有牵连还会涉及到同步与互斥相关技术；

不同点：

1）多进程占用系统资源较多，多线程占用系统资源较少；

2）每个进程拥有独立且完整的用户空间，每个线程自己只占用少量的栈空间，同时依赖所处进程的资源空间；

3）进程因为其独立性，所以进程间通信需要依赖一个公共的媒介（文件/内存），同进程下的多个线程公用进程资源，不存在通信这一说，但是要防止线程之间相互竞争资源（互斥/同步）；

4）多个进程间协同工作主要依赖通信及同步机制，多个线程间协同工作主要依赖互斥及同步机制（而且进程与线程同步互斥的控制方法完全不同）；

内存划分为哪几块区域：

- **用户空间**：包括代码段（存放操作指令）、data数据段（存放已初始化的全局变量）、BSS 段（存放未初始化的全局变量）、堆（存放动态分配的内存）、栈（存放临时创建的局部变量和函数调用上下文信息）等。用户空间是私有的，每个进程都有自己的用户空间。对于线程来说只有栈区是自己独立的内存空间，其他区域都是共享的。
- **内核空间**：内核空间是共享的，所有进程都可以访问。内核空间包含内核代码和数据结构，用于实现操作系统的核心功能。

4.6 如何理解线程，有哪些常用的多线程编程模型

线程是轻量级的进程，多个线程共享进程的资源；多线程代码需要分为多条线路同时考虑其运行效果，除了让线程执行函数完成既定的任务外，往往需要考虑线程之间的互斥问题。

常用的多线程编程框架有：

1）生产者消费者模型：生产线程负责产生数据，消费线程负责使用数据，当数据使用完毕时就停止消费，等待生产线程生产数据。这种模型下，消费线程依赖生产线程，并且生产线程往往只有 1 个；

2）主从模型：主线程负责接收任务（网络数据包，用户交互等等），并分析任务需要的处理函数，启用工作线程调用相应的函数处理该任务。该模式下主线程交互周期非常短，保证了了处理并发任务的速度；

3）线程池模型：提前创建一堆休眠线程（休眠在某些锁或者条件变量上面），只留下管理类的几个线程活跃。当管理线程发现有任务时，随机激活单个休眠线程去处理任务；而当所有休眠线程均处于活跃状

华清远见武汉中心嵌入式学科面试宝典

态时，管理线程还可以创建新的休眠线程；当太多休眠线程休眠的时候，管理线程还能杀死部分休眠线程。线程池模型可以说是主从模型中的一种的特例。

4.7 死锁是什么概念，一般是如何产生的

死锁 (deallocks)：是指两个或两个以上的进程（线程）在执行过程中，因争夺资源而造成的一种互相等待的现象，若无外力作用，它们都将无法推进下去。此时称系统产生了死锁，这些永远在互相等待的进程（线程）称为死锁进程（线程）。

产生死锁的条件：

1）互斥条件：一个资源每次只能被一个进程（线程）使用。

2）请求与保持条件：一个进程（线程）请求资源而阻塞时，对已获得的资源保持不放。

3）不剥夺条件：此进程（线程）已获得的资源，在末使用完之前，不能强行剥夺。

4）循环等待条件：多个进程（线程）之间形成一种头尾相接的循环等待资源关系。

死锁举例：

两个/多个进程（或线程）在借助信号量访问临界资源时，若进程 A 正常执行需要资源 1 和资源 2，没拿到足够的资源而阻塞，恰好另外一个进程 B 正常执行也需要资源 1 和资源 2，而 B 也因为资源不够而阻塞；明明进程 B/A 放弃自己占用的部分资源就能让对方先运行起来，但是谁都没有放弃自己占用的资源。这时就会出现死锁。

如何避免死锁：

避免死锁的产生只需要打破以上任意一个条件即可，其中第一个条件不可打破。

1、打破请求与保持条件：资源分配策略上采取措施，给资源设定优先级，申请顺序必有由高到低，或者将多个资源整合成一个资源。

2、打破不剥夺条件：使用可剥夺资源，允许关键资源共享。

3、打破循环等待条件：使用顺序锁或轮询锁，确保资源正确获取和释放。

如果无法打破条件，产生死锁后如何解除有以下方式：

检测死锁：当死锁发生时，系统能够检测到并采取措施修复，如重启服务或终止线程。

超时重试：设置超时机制，当进程等待资源超时时，释放已占有的资源。

4.8 五种 IO 模型 并说明是同步还是异步

华清远见武汉中心嵌入式学科面试宝典

阻塞式 I/O：是最流行的 I/O 模型，默认的情况下所有套接字都是阻塞的。

非阻塞式 I/O：设置成非阻塞是通知进程，当所请求的 I/O 操作非得把本进程置于休眠状态才能完成时，则不要置于休眠状态，而是返回一个错误。

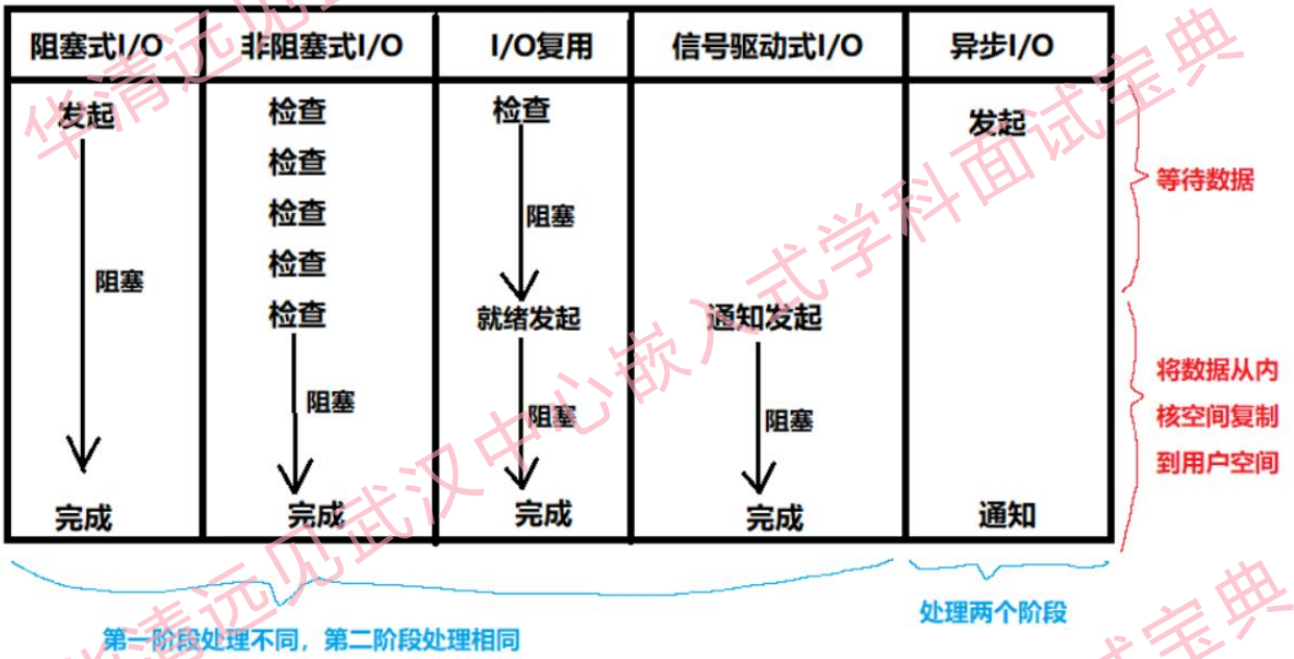
I/O 多路复用：同时阻塞监听多个 I/O 操作，任意一个或多个触发后都会继续执行。

信号驱动式 I/O：内核在描述符就绪时会发送 SIGIO 信号通知。

异步 I/O：给内核传递描述符、缓冲区指针、缓冲区大小和文件偏移，并告知内核再操作完成后如何通知我们。

同步 I/O 操作指会导致阻塞进程，直到 I/O 操作完成的操作。

异步 I/O 操作不会导致请求阻塞。其中除了异步 I/O 外的其它四种都是同步 I/O。



4.9 线程同步互斥的具体实现方法

互斥：**一个线程访问资源的时候，不允许其他线程访问**

实现互斥可以使用互斥锁，将临界区锁住即可

使用互斥锁需要先定义并初始化，上锁与解锁函数分别为 pthread_mutex_lock() 和 pthread_mutex_unlock()，用完后需要销毁互斥锁 pthread_mutex_destroy()。

临界资源：多个线程/进程共享的资源，且同一时刻只能有一个人使用的资源

临界区：访问临界资源的代码

华清远见武汉中心嵌入式学科面试宝典

同步：线程间相互配合，按照一定的顺序完成任务

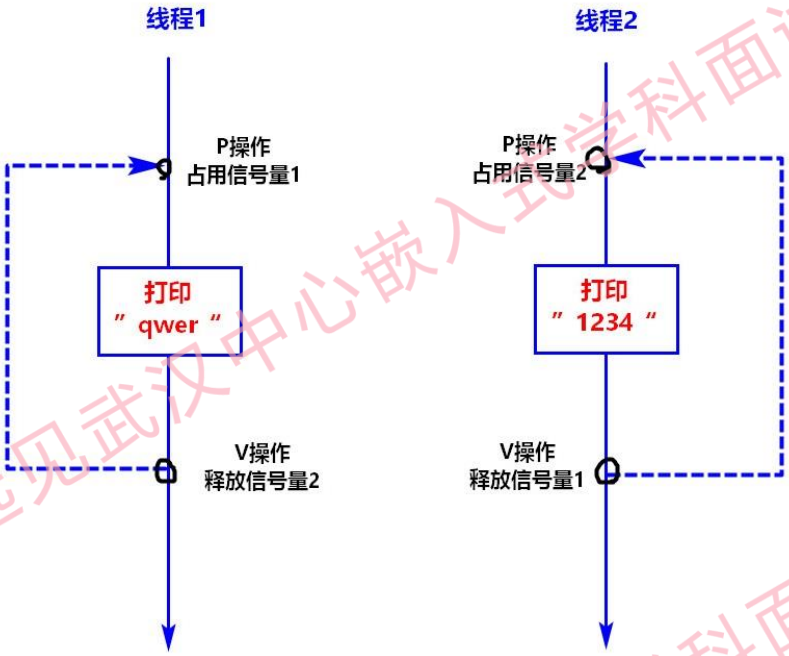
实现同步可以使用**条件变量**或**信号量**实现

条件变量是**利用线程间共享的全局变量进行同步通信**的一种机制。多个线程**等待某个条件满足**，在条件达到时**由一个线程去通知其他等待的线程** 条件变量总是和互斥锁结合在一起使用。

实现流程是**先用互斥锁将临界区锁住，在使用资源时进行判断，如果不满足条件则使用条件变量进行阻塞 pthread_cond_wait()，该函数在阻塞的 释放掉互斥锁**，此时其他线程拿到资源后开始执行，**另一个线程执行结束后唤醒该阻塞的线程即可，** pthread_cond_broadcast()函数唤醒所有等待的线

信号量是一个与队列有关的整型变量，实现同步的方式主要有 P 操作与 V 操作。

实现流程是**先分配资源，每个线程从自己的资源区获取资源，只有拿到资源才可以执行**，申请资源使用的则是 P 操作 sem_wait()，**执行结束后将该 到其它线程的资源区**，也就是 V 操作 sem_post()。



4.10 可重入函数与不可重入函数

可重入函数：函数重入是一种现象，说的是函数执行到一半时，被所在的线程/进程在信号处理函数或者其他原因再次调用的现象。递归本质上也是函数重入，信号处理函数被信号打断而重新调用也算重入。函数重入如果发生且函数依旧能返回正确的结果，就说这个函数是可重入的。注意可重入是一种函数属性。

可以这样理解，重入即表示重复进入，首先它意味着这个函数可以被中断，其次意味着它除了使用自己栈上的变量以外不依赖于任何环境（包括 static），这样的函数就是 purecode（纯代码）可重入，可以允许有多个该函数的副本依赖资源压栈，由于它们使用的是分离的栈，所以不会互相干扰。

华清远见武汉中心嵌入式学科面试宝典

不可重入函数：如果函数内需要访问全局变量或者有使用静态局部变量，那么该函数重入会影响上一次调用的结果（不可重入函数）。不可重入函数在实时系统设计中被视为不安全函数，在设计底层代码时需要考虑函数被中断或者被意外的压栈后再次进入的情况。

总结：可重入和不可重入是一个底层概念，常见于递归、多线程、中断等场景，同时也可以说它是一种函数属性，即函数自己没有执行完成能否再次调用函数自己，而且返回后早先的调用结果依旧正确；

4.11 操作系统中进程调度策略有几种？以及 Linux 的任务调度机制

调度策略：

1．先来先服务（FCFS）：按照进程到达的顺序进行调度，先到达的进程先被执行。

2．短作业优先（SJF）：选择执行时间最短的进程优先执行，以减少平均等待时间。

3．优先级调度：根据进程的优先级来决定执行顺序，优先级高的进程先执行。

4．时间片轮转（Round Robin）：每个进程被分配一个时间片，当时间片用完时，进程被暂停并放到队列末尾，轮流执行直到所有进程完成。

5．多级反馈队列调度：将进程按照优先级划分到不同的队列，每个队列有不同的时间片大小，进程根据优先级在不同队列中执行。

6．最短剩余时间优先（SRTF）：在 SJF 的基础上，动态调整执行中的进程，如果有新进程到达且其执行时间比当前进程剩余时间短，则切换执行新

任务调度机制：

Linux 的任务调度机制主要涉及进程调度，将进程分为不同类别以适应不同的需求，包括**交互式进程、批处理进程和实时进程**。此外，Linux 还支持 程和普通进程的调度，其中**实时进程应优先于普通进程运行**。Linux 中的任务调度机制包括多种调度策略，如 FIFO（先来先服务）、RR（时间片轮 等，每个进程有两个优先级（动态优先级和实时优先级），而实时进程则有第三种优先级，即实时优先级。

交互式进程：这类进程涉及大量的人机交互，需要不断处于睡眠状态等待用户输入，对系统响应时间要求较高，如编辑器等。

批处理进程：不需要人机交互，在后台运行，可以占用大量系统资源，但能忍受较长的响应延迟，如编译器。

实时进程：对调度延迟的要求最高，执行重要操作需要立即响应，如视频播放软件或飞机飞行控制系统。实时进程采用 FIFO 或 RR 调度策略，具有 级，确保及时响应。

华清远见武汉中心嵌入式学科面试宝典

Linux 中的调度器根据进程的类型和优先级决定哪个进程应该获得 CPU 时间。例如，CFS（Completely Fair Scheduler）调度器主要用来调度普通任务 任务则采用其他调度策略如 EDF（最早截止时间优先）等。这种机制确保了系统能够高效地管理和分配资源，满足不同类型进程的需求。

4.12 进程的状态以及概念

基本状态有三种：就绪态、运行态、等待态。其中等待态包含可中断等待态和不可中断等待态。

就绪态：进程所有需要的条件已经准备完成，等待 CPU 的调度

运行态(R)：进程占用 CPU，并在 CPU 上运行

可中断等待态(S)：进程正在休眠，等待某个资源来唤醒它。也可以被其他信号中断唤醒

不可中断等待态(D)：进程正在休眠，等待某个资源来唤醒它。不能被其他信号中断唤醒

补充其它状态：

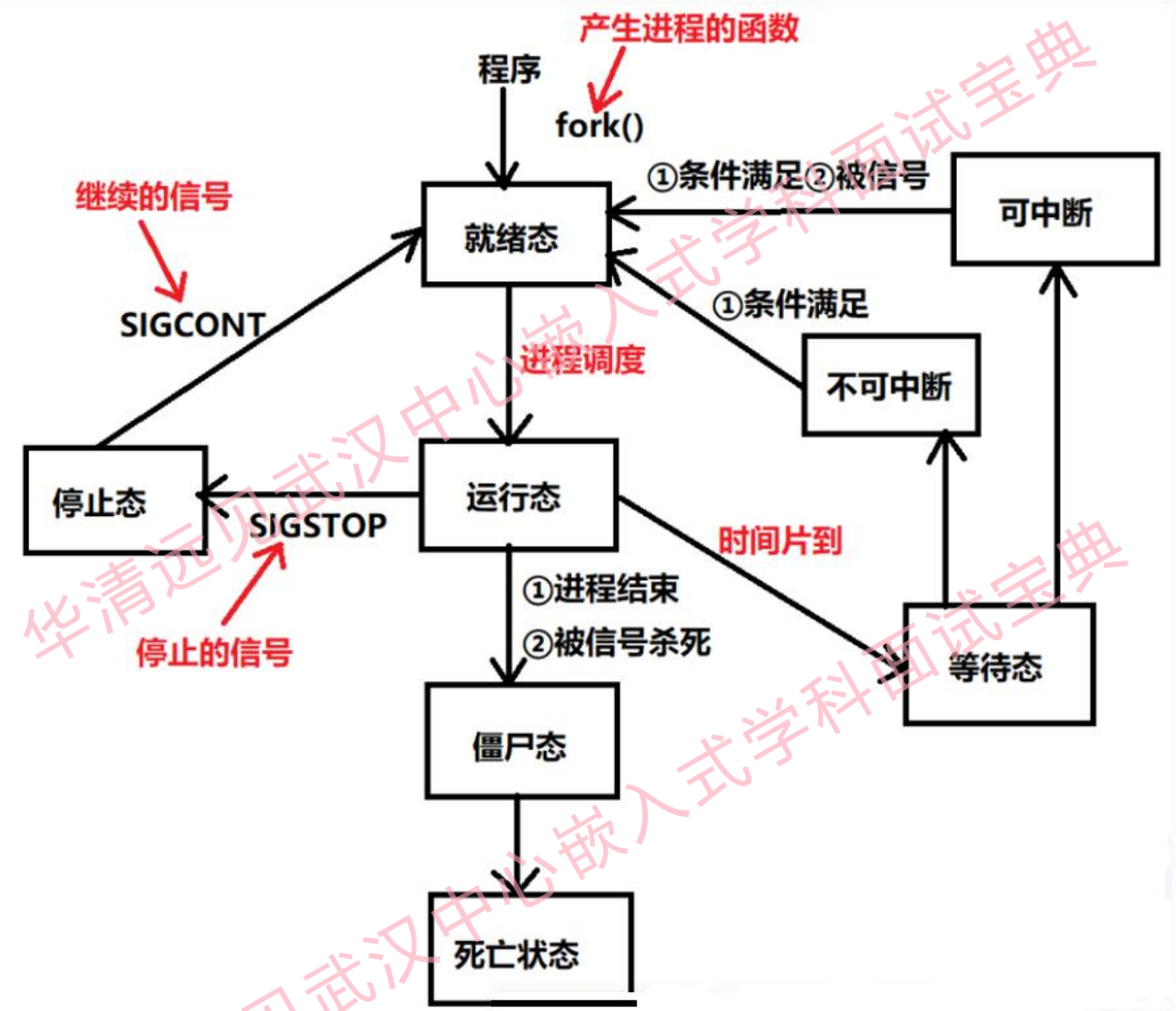
停止态(T)：进程暂停接受某种处理。例如：gdb 调整断点信息处理

僵尸态(Z)：进程已经结束但还没有释放进程资源。例如：PID，PPID 未释放

退出态(X)：也被称为死亡状态，进程即将被销毁，状态时间很短，几乎不会被捕捉到

状态转换图如下：

华清远见武汉中心嵌入式学科面试宝典



4.13 线程的属性以及区别，如何设置线程的属性

线程的属性分为结合式属性与分离式属性。在任何一个时间点上，线程是可结合的（joinable），或者是分离的（detached）。**一个可结合的线程 他线程收回其资源和杀死；在被其他线程回收之前，它的存储器资源（如栈）是不释放的。相反，一个分离的线程是不能被其他线程回收或杀死的，它 资源在它终止时由系统自动释放。**

程属性默认是结合式，设置为分离式可以在创建线程时，配置 pthread_create()函数的第二个参数，也可以在创建子线程后调用 pthread_deta 为分离式线程。

华清远见武汉中心嵌入式学科面试宝典